

TEAM AURIX · TURTLEBOT3 · ROS2 HUMBLE · PYQT5

TurtleBot3 PyQt5 대시보드

로봇 상태 모니터링 · 수동/자율 주행 제어를 하나로 통합한 데스크톱 대시보드

burger 모델 · ROS2 Humble · Ubuntu 22.04

팀원 도희정 · 최치언

CONTENTS

목차

- | | | | |
|----|--------------|----|----------------------------------|
| 01 | 개요 | 05 | 각 영역별 기능 설명 |
| 02 | 개발 환경 | 06 | 사용 방법 및 플로우 |
| 03 | 기술 스택 | 07 | 추가 기능 (SSH · 자동 Bringup · Voice) |
| 04 | GUI 전체 화면 구성 | 08 | 트러블슈팅 |

01. OVERVIEW

개요

TurtleBot3(burger)의 상태 확인과 제어가 터미널 명령·별도 teleop 도구로 흩어져 있던 문제를, PyQt5 GUI 하나로 통합했습니다.

실시간 상태 모니터링

연결 상태(ODOM/BATTERY/LiDAR), 배터리 잔량·전압, 위치·속도(odom), LiDAR 최소 거리를 상단에 항상 고정 표시

수동 + 자율 주행 통합

방향 버튼/키보드(WASD)로 수동 조작하거나, 지도에서 만든 waypoint로 Nav2 기반 자율 주행(단일 목적지·경로) 실행

원격 운영 지원

SSH로 로봇에 접속해 bringup·음성 안내 노드를 대시보드에서 바로 실행. 대시보드를 꺼도 로봇 프로세스는 계속 동작

다중 로봇 안전장치

로봇마다 ROS_DOMAIN_ID를 분리해서, 여러 로봇이 같은 명령을 동시에 받는 사고를 방지

02. DEVELOPMENT ENVIRONMENT

개발 환경

운영체제	Ubuntu 22.04 LTS (Jammy Jellyfish)
로봇 미들웨어	ROS2 Humble Hawksbill
언어	Python 3.10
GUI 프레임워크	PyQt5 5.15 (Qt 5.15)
로봇 플랫폼	TurtleBot3 Burger (OpenCR 보드 + LDS-03 LiDAR)
자율주행 스택	Nav2 (turtlebot3_navigation2 패키지 기반)

03. TECH STACK

기술 스택

영역	기술	설명
GUI	PyQt5	메인 윈도우, 탭/위젯, 전역 QSS 테마(theme.py)
로봇 통신	rclpy (ROS2 Humble)	odom/battery_state/scan/amcl_pose 구독, cmd_vel 발행
자율 주행	Nav2 액션	NavigateToPose, FollowWaypoints (nav2_msgs)
속도 제어	nav2_msgs/SpeedLimit	controller_server에 런타임 속도 제한 발행
원격 실행	Paramiko (SSH)	원격 bringup/음성 노드 실행, 계정·도메인 지정 접속
데이터 저장	PyYAML	waypoint/trajectory YAML 저장·불러오기
지도 처리	Pillow (PIL)	nav2 map_server YAML+PGM/PNG 로더, 좌표 변환
메시지 타입	ROS2 표준 메시지	Twist, Odometry, BatteryState, LaserScan 등

03. TECH STACK

프로젝트 구조

```
main.py
dashboard/
  ros_bridge.py
  main_window.py
  theme.py
  map_loader.py / waypoints.py
widgets/
  status_panel.py
  control_panel.py
  map_panel.py
  nav_panel.py
  log_panel.py
  ssh_panel.py
launch/dashboard_bringup.launch.py
config/ , maps/
```

진입점 (QApplication, 전역 스타일시트 적용)

rclpy 노드 ↔ Qt 시그널 브리지 (백그라운드 스레드)
 메인 윈도우, 탭 구성, 위젯 간 시그널 연결
 팔레트 · QSS 전역 테마
 지도 · waypoint YAML 로더

상단 로봇 상태 바
 동작 컨트롤 탭
 맵 / 웨이포인트 생성 탭
 웨이포인트 주행 탭
 이벤트 로그 탭

SSH 연결 패널 (상단 고정)
 Nav2 + 대시보드 통합 실행
 waypoint YAML , nav2 지도 파일

04. GUI LAYOUT

GUI 전체 화면 구성

모든 탭에서 항상 보이는 상단 고정 영역 + 목적별 4개 탭 구조

① SSH 연결 패널 (Host / Username / Password / Domain ID · 연결 상태)

② 헤더 (모델명 · ROS_DOMAIN_ID · 전체 토픽 수신 상태)

③ 로봇 상태 바 (연결 배지 · 배터리 · LiDAR 거리 · X/Y/Yaw · 선속도/각속도)

동작 컨트롤

맵 / 웨이포인트 생성

웨이포인트 주행

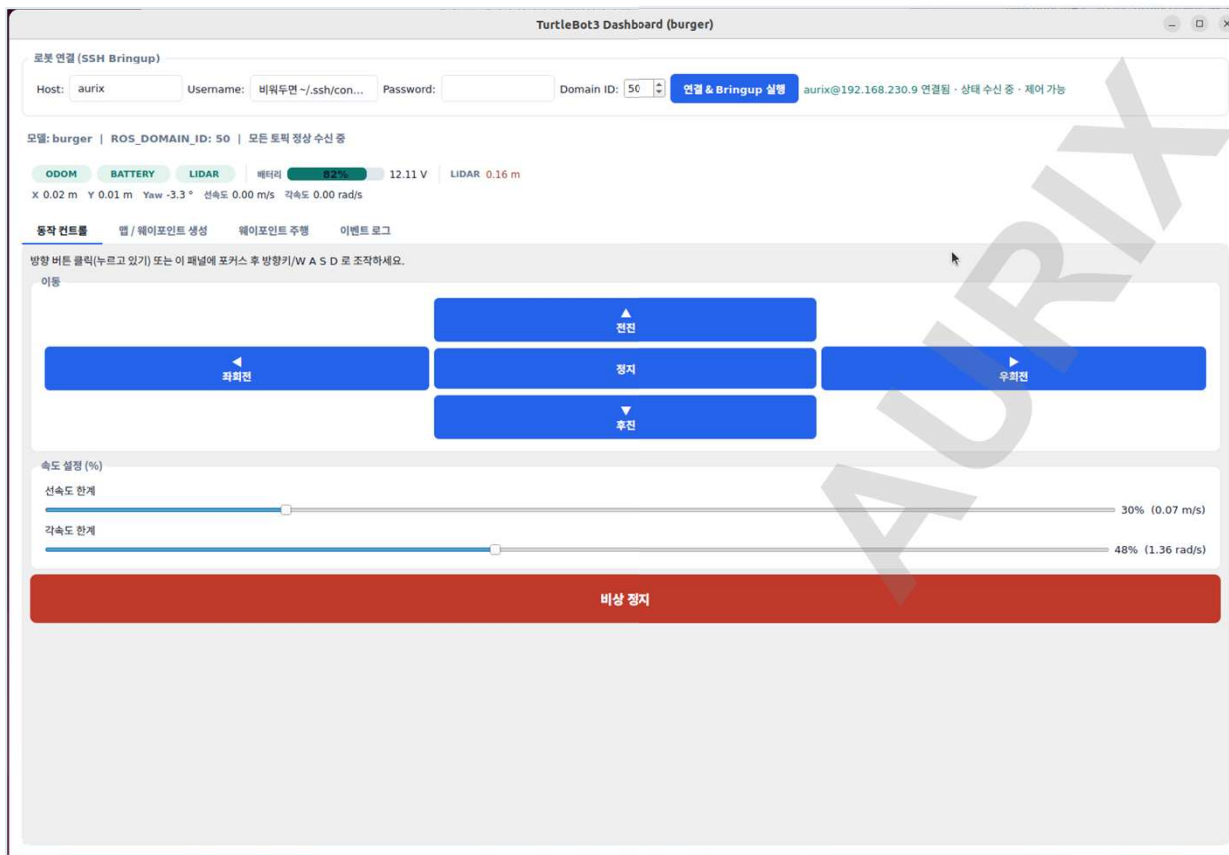
이벤트 로그

④ 탭 콘텐츠 영역 — 선택한 탭에 따라 전환

(디자인: 화이트 캔버스 + 블루 포인트 컬러의 미니멀 스타일, theme.py에서 전역 관리)

05. FEATURES · 동작 컨트롤

화면 구성 — 동작 컨트롤 탭



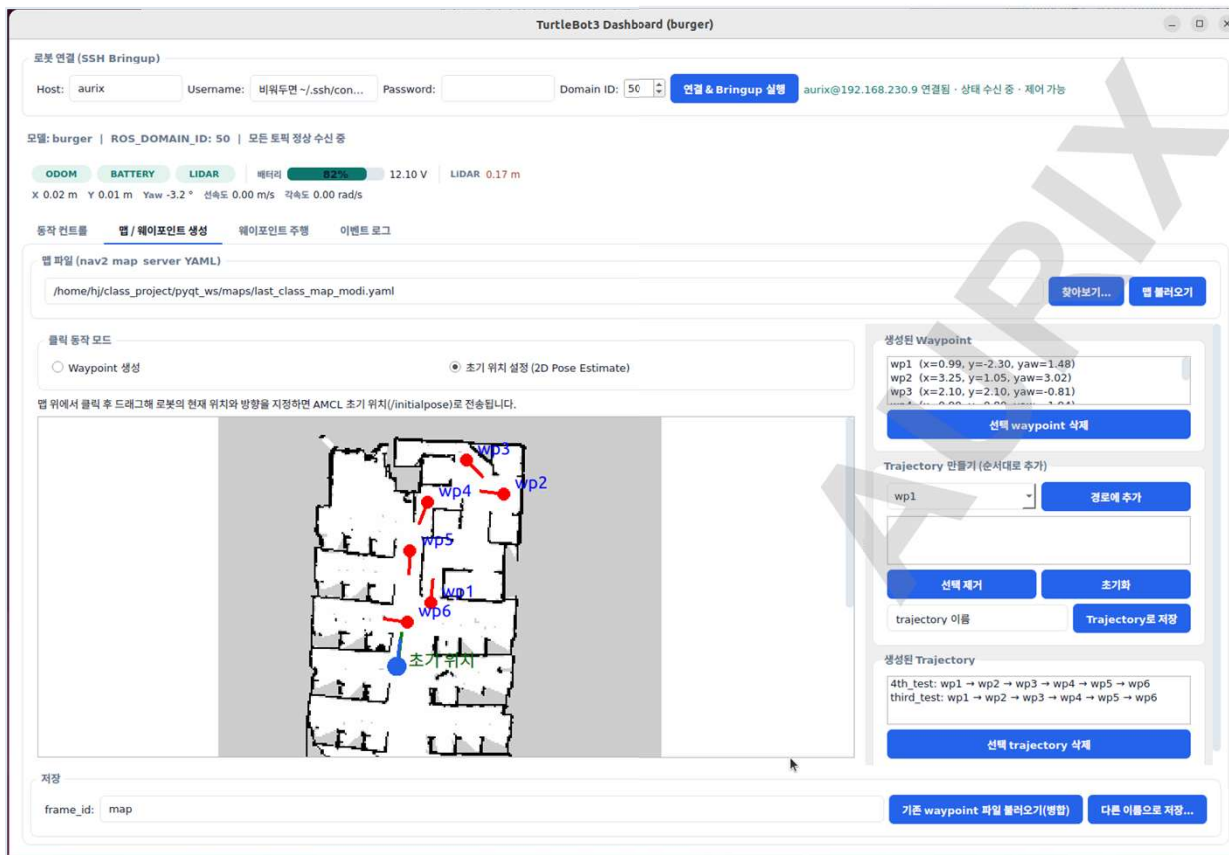
— **방향 버튼 / 키보드 조작** — 버튼 클릭(누르고 있기) 또는 방향키·WASD로 수동 주행

— **속도 설정(%)** — 선속도·각속도 슬라이더로 최대 속도를 퍼센트로 제한

— **비상 정지** — 즉시 속도 0으로 정지 + 이벤트 로그에 경고 기록

05. FEATURES · 맵/웨이포인트 생성

화면 구성 — 맵 / 웨이포인트 생성 탭



TurtleBot3 PyQt5 Dashboard

— 지도 불러오기 — nav2 map_server
YAML(+PGM/PNG) 로드

— Waypoint 생성 — 클릭 = 제자리(yaw 0),
클릭+드래그 = 방향 지정

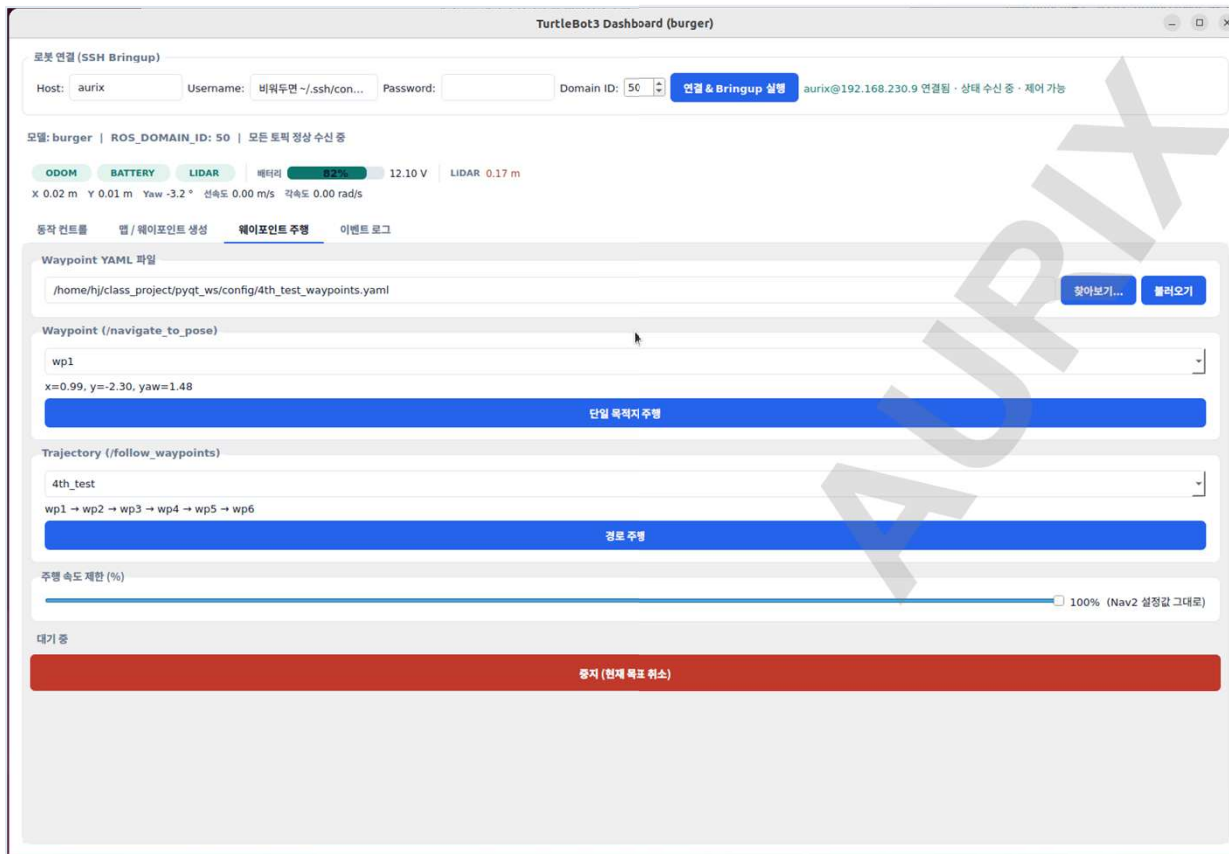
— 2D Pose Estimate — 클릭+드래그로
AMCL 초기 위치(/initialpose) 전송

— Trajectory 구성 — waypoint를 순서대로
묶어 경로로 저장

— 저장/불러오기 — YAML로 저장, 기존 파
일 병합 불러오기 지원

05. FEATURES · 웨이포인트 주행

화면 구성 — 웨이포인트 주행 탭

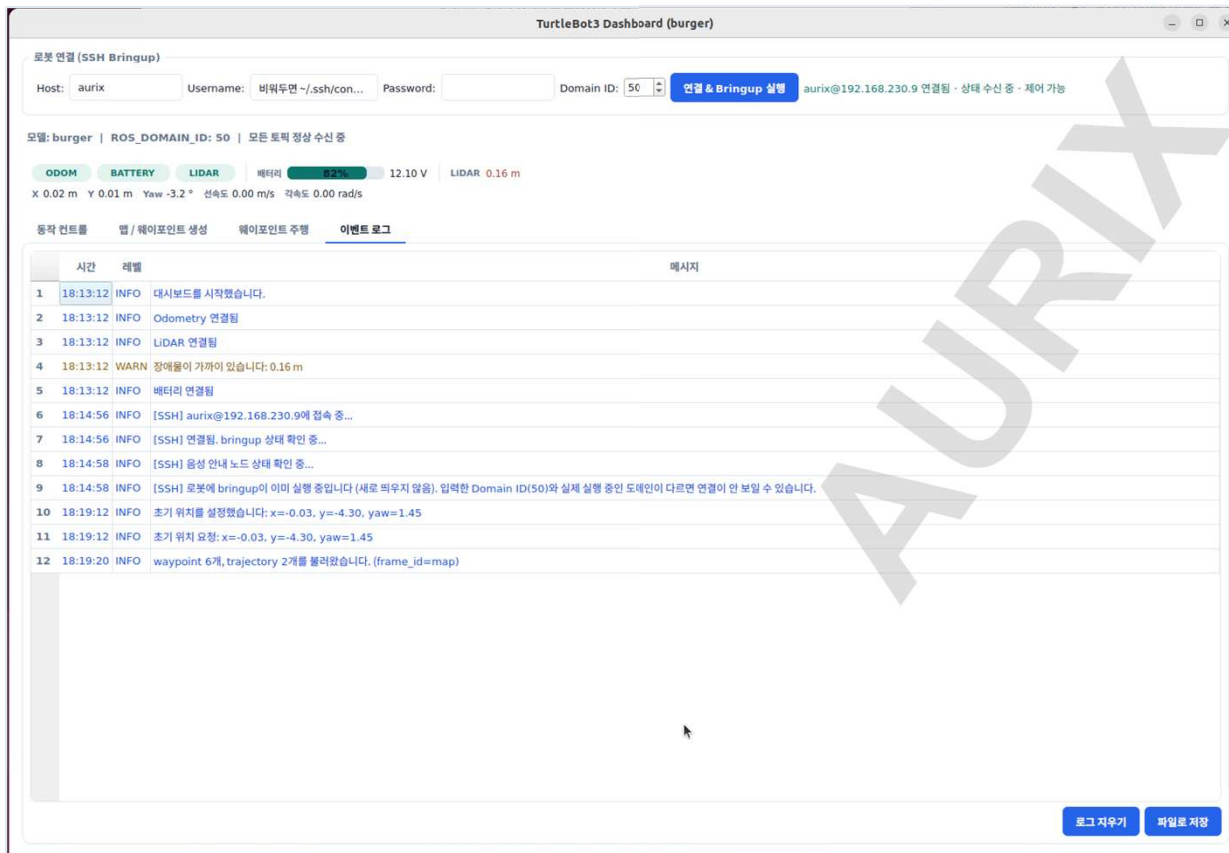


TurtleBot3 PyQt5 Dashboard

- **YAML 불러오기** — waypoint/trajectory 파일 선택 및 불러오기
- **단일 목적지 주행** — /navigate_to_pose 액션으로 목표 전송
- **경로 주행** — /follow_waypoints 액션으로 순차 주행
- **주행 속도 제한(%)** — Nav2 파라미터 파일을 고치지 않고 런타임으로 속도 조절
- **중지** — 현재 목표를 즉시 취소

05. FEATURES · 이벤트 로그

화면 구성 — 이벤트 로그 탭



— 시간순 기록 — 연결 상태 변화, 저배터리/장애물 경고, SSH 진행상황

— 주행 이벤트 — waypoint 도달, 목적지 성공/취소/실패 결과

— 로그 관리 — 지우기 / 텍스트 파일로 저장

05. FEATURES

각 영역별 기능 설명 — 로봇 상태 바

- **연결 배지 (ODOM · BATTERY · LiDAR)** — 정상(초록) / 불안정(빨강) / 끊김(회색) 3단계. 위치독 타임아웃(3초 끊김 / 1.5초 불안정) 기준
- **배터리** — 전압 기반 % 환산(3S LiPo 12.6V=100% ~ 9.9V=0%). 시간상수 기반 저역통과 필터(5초) + 표시 히스테리시스($\pm 1.25\%$)로 떨림 방지
- **LiDAR 최소 거리** — 장애물 임계값(0.3m) 이하일 때 빨간색으로 강조, 이벤트 로그에 경고 기록
- **위치 / 속도 (odom)** — X, Y, Yaw와 선속도·각속도. 정지 시 -0.00 표시 떨림을 막는 0값 정규화 적용

05. FEATURES

각 영역별 기능 설명 — SSH 연결 패널

- **Host / Username / Password** — 접속 대상 IP(또는 ~/.ssh/config 별칭) · 로그인 계정 · 비밀번호
- **Domain ID** — 이 로봇의 ROS_DOMAIN_ID. 로봇마다 다르게 지정해 서로 격리
- **연결 & Bringup 실행** — SSH 접속 → bringup 상태 확인 → 필요 시 원격 실행까지 한 번에
- **상태 라벨** — "SSH 버튼을 눌렀는지"가 아니라 실제 로봇 상태 수신·제어 가능 여부를 실시간으로 표시

06. USAGE

사용 방법 — 실행

방법 A — Nav2 + 대시보드 한 번에 (권장)

```
cmd> ./run.sh
```

launch/dashboard_bringup.launch.py를 호출해
Nav2(turtlebot3_navigation2)를 먼저 띄우고
몇 초 뒤 대시보드를 자동으로 함께 실행합니다.

옵션 예시

```
./run.sh map:=/path/to/map.yaml
```

```
./run.sh use_rviz:=true
```

방법 B — 대시보드만 실행

```
cmd> python3 main.py
```

Nav2가 이미 별도로 떠 있거나, 웨이포인트
주행 없이 모니터링/수동 조작만 쓸 때 사용합니다.

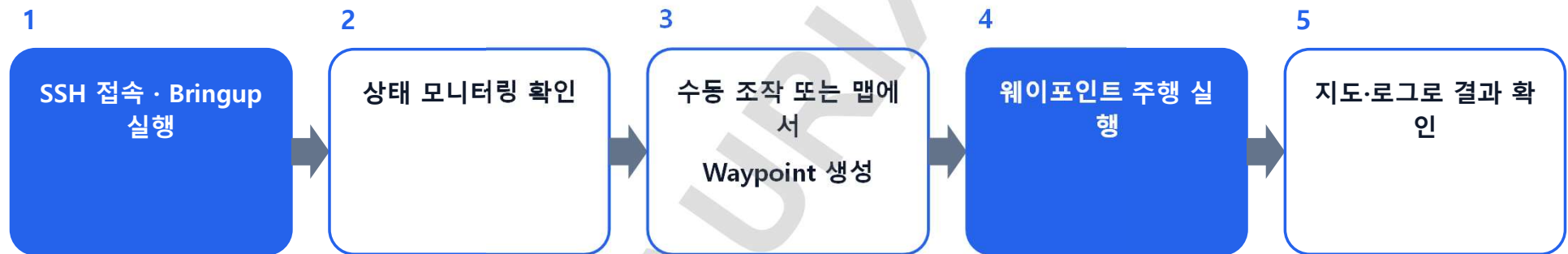
필요 환경변수

TURTLEBOT3_MODEL (기본 burger)

ROS_DOMAIN_ID

06. USAGE

전체 사용 플로우



- SSH 패널에서 Host/Username/Password/Domain ID 입력 후 접속 → 필요 시 자동 bringup
- 상단 상태 바에서 ODOM/BATTERY/LiDAR 연결 및 배터리 상태 확인
- 동작 컨트롤 탭에서 수동 조작, 또는 맵/웨이포인트 생성 탭에서 목표 지점 생성
- 웨이포인트 주행 탭에서 단일 목적지 또는 경로 선택 후 주행 (지도에 목표·현재 위치 실시간 표시)
- 이벤트 로그 탭에서 성공/실패/경고 이력 확인, 필요 시 파일로 저장

07. EXTRA FEATURES

추가 기능 — SSH 원격 접속 & 로봇별 격리

대시보드 안에서 로봇에 직접 접속하고, 여러 로봇을 오가도 서로 명령이 섞이지 않도록 격리합니다.

- **원격 접속** — Host/Username/Password만 있으면 어떤 로봇에도 SSH 접속 (별칭 없이도 계정명 직접 지정 가능)
- **Domain ID 지정 접속** — 접속할 로봇의 ROS_DOMAIN_ID를 지정 — bringup 실행 시 그 도메인으로 원격 실행
- **대시보드 자체 재연결** — 접속 성공 시 대시보드의 ROS2 노드도 그 도메인으로 다시 시작해, 이후로는 그 로봇의 토픽만 보이고 제어됨
- **대상 로봇 표시** — 상태 라벨에 마지막으로 접속 성공한 계정@호스트를 계속 표시해 혼동 방지

07. EXTRA FEATURES

추가 기능 — 자동 Bringup

로봇에 SSH로 접속하면, bringup이 켜져 있는지 자동으로 확인하고 필요할 때만 실행합니다.

- **중복 실행 방지** — pgrep으로 turtlebot3_bringup 실행 여부 확인 → 이미 떠 있으면 새로 띄우지 않음 (모터/LiDAR 충돌 방지)
- **여러 팀원 동시 접속 안전** — 누가 먼저 띄웠든 상관없이 항상 같은 규칙으로 동작
- **세션과 분리 실행** — setsid + nohup + disown으로 실행 — SSH 세션이 끊기거나 대시보드를 꺼도 로봇의 bringup은 계속 동작
- **실패 시 확인** — 원격 bringup 로그는 로봇의 ~/bringup_dashboard.log 에서 확인 가능

07. EXTRA FEATURES

추가 기능 — Voice 음성 안내 자동 실행

bringup과 함께 음성 안내 노드를 자동으로 확인·실행하고, 주요 주행 이벤트를 음성으로 안내합니다.

- 음성 노드 자동 확인/실행 — tb3_voice_notifier가 안 떠 있으면 bringup과 같은 ROS_DOMAIN_ID로 자동 실행
- 동작 가능 안내 — bringup을 새로 띄운 경우 "로봇 동작이 가능합니다" 음성 안내 발행
- 주행 이벤트 안내 — 주행 시작 · waypoint 도달 · 목적지 완료 등 주요 이벤트를 음성으로 실시간 안내

08. TROUBLESHOOTING

트러블슈팅

개발 과정에서 발견·해결한 주요 이슈들을 Git 커밋 히스토리 기준으로 정리했습니다.

- 배터리 % 표시 떨림
- 정지 상태 각속도 0.00 / -0.00 떨림
- SSH 상태 라벨이 실제 로봇 상태와 무관하던 문제
- 웨이포인트 주행 목표가 지도에 안 보이던 문제
- 지도 위 로봇 위치 — odom 대신 amcl_pose 사용
- 여러 로봇이 동시에 같은 명령을 받던 문제
- 주행 중 배터리 41% 이하에서 odom/battery 연결 끊김 (조사 중)

08. TROUBLESHOOTING · 1

배터리 % 표시 떨림

증상

배터리 진행바 퍼센트가 값이 계속 미세하게 바뀌며 깜빡임 (예: 81% ↔ 83%)

원인

전압 EMA가 "메시지 횟수" 기준이라 토픽 발행이 빠르면 필터링이 거의 안 됨 + 정수 반올림 경계에서 표시값이 왕복

해결

경과 시간 기반 저역통과 필터(시정수 5초) 적용 + 표시값에 히스테리시스($\pm 1.25\%$) 추가

08. TROUBLESHOOTING · 2

정지 상태 각속도 0.00 / -0.00 떨림

증상

로봇이 완전히 정지해 있는데도 각속도 표시가 0.00과 -0.00 사이를 왕복

원인

센서 노이즈로 값이 0 근처에서 미세하게 +/-를 오가면 반올림 결과가 음수 0(-0.0)이 되어 부호만 다르게 표시됨

해결

status_panel에 `_fmt_zero_safe()` 헬퍼 추가 — 반올림 후 0이면 부호를 정규화 (x/y/yaw/선속도/각속도 전체 적용)

08. TROUBLESHOOTING · 3

SSH 상태 라벨과 실제 로봇 상태 불일치

증상

로봇이 이미 켜져 있고 정상 동작 중인데도 SSH 패널에는 "아직 연결 안 됨"으로 표시되어 오해 유발

원인

상태 라벨이 "이번 세션에서 SSH 버튼을 눌렀는지"만 반영할 뿐, 실제 ROS 토픽 수신과는 무관하게 설계되어 있음

해결

main_window가 odom/battery/lidar 수신 상태를 종합해 실시간으로 라벨에 반영 (정상/일부 지연/끊김 3단계)

08. TROUBLESHOOTING · 4

웨이포인트 주행 목표가 지도에 안 보임

증상

웨이포인트 주행 탭에서 선택한 목적지가 맵/웨이포인트 생성 탭 지도에는 표시되지 않아, 로봇이 맞게 가는지 확인 불가

원인

두 탭이 각자 독립적인 waypoint 데이터를 관리해서, 주행 탭에서 불러온 파일이 지도 탭에 자동으로 반영되지 않았음

해결

nav_panel이 불러온 waypoint를 지도에 자동 동기화 + 현재 주행 목표는 주황색으로 강조 표시 (경로 주행 시 다음 지점으로 함께 이동)

08. TROUBLESHOOTING · 5

지도 위 로봇 위치 표시 — odom 대신 amcl_pose

증상

실시간 로봇 위치를 지도에 표시하고 싶었지만, 어떤 좌표를 써야 정확한지가 관건이었음

원인

지도는 'map' 좌표계인데 /odom은 시간이 지나며 map과 어긋남(드리프트) — 그대로 지도에 얹으면 실제 위치와 달라 보일 수 있음

해결

Nav2 AMCL이 추정해 온 /amcl_pose(map 좌표계)를 구독해 표시. AMCL이 꺼져 있으면 마커가 안 보일 뿐 오류로 처리하지 않음

08. TROUBLESHOOTING · 6

여러 로봇이 동시에 같은 명령을 받는 문제

증상

SSH로 다른 로봇에 접속했는데, 원래 연결돼 있던 로봇도 함께 움직임 — 대시보드는 로봇 하나만 제어해야 함

원인

ROS2/DDS 통신은 IP가 아니라 ROS_DOMAIN_ID 단위로 범위가 정해짐. 두 로봇이 같은 도메인이면 모든 토픽/액션을 서로 다 주고받음

해결

SSH 패널에 Domain ID 필드 추가 — 로봇마다 다른 도메인 지정 + 접속 성공 시 대시보드의 ROS2 노드 자체를 그 도메인으로 재시작

* 이 항목은 이번 세션에서 구현하여 실제 동작까지 확인한 가장 최신 개선 사항입니다.

08. TROUBLESHOOTING · 조사 중

주행 중 배터리 연결 끊김 (하드웨어 이슈)

증상

배터리 41% 이하 구간에서, 주로 주행 중에 ODOM·BATTERY 연결만 끊기고 LiDAR는 계속 정상

분석

41%는 전압으로 약 11.0V — 3S LiPo 기준 절대적 저전압은 아님. odom·battery는 OpenCR 한 보드에서, LiDAR는 별도 USB 경로라 이 패턴이면 OpenCR 쪽 전원/연결 문제로 추정. 속도를 50%로 낮춰도 재현되어, 순수 부하량보다 배터리 노화(내부저항) 또는 진동에 의한 커넥터 접촉 불량 가능성이 유력

다음 단계

dmesg로 USB(OpenCR) 재연결 여부 확인 → 다른 배터리로 교체 재현 테스트 → 배터리/전원 커넥터 점검

* 소프트웨어(대시보드) 코드로는 재현되지 않는 패턴이라, 하드웨어/전원 이슈로 분류하여 별도 확인 중입니다.

SUMMARY

모니터링 · 수동 조작 · 자율 주행 · 원격 운영을 하나의 대시보드로

- 실시간 상태 모니터링과 수동/자율 주행 제어를 하나의 PyQt5 앱으로 통합
- 지속적인 트러블슈팅으로 표시 안정성(배터리·속도), 정보 일치성(SSH 상태, 지도 동기화) 개선
- ROS_DOMAIN_ID 기반 로봇별 격리로 다중 로봇 환경에서도 안전하게 운영
- 모든 변경 이력은 Git 커밋으로 관리 — 트러블슈팅 과정도 커밋 메시지에 함께 기록

TEAM AURIX · TURTLEBOT3 · ROS2 HUMBLE · PYQT5

감사합니다.

로봇 상태 모니터링 · 수동/자율 주행 제어를 하나로 통합한 데스크톱 대시보드
